

Abouelfetouh Ashraf Moussa

QUIZ Golang

=====

What is the output:-

1-

```
m := map[string]int{"a": 1}
v, ok := m["b"]
fmt.Println(v, ok)

v2 := m["b"]
fmt.Println(v2 + 10)
```

0 false

10

"b" key not found

but v2 take default value = 0

2-

```
a := make([]int, 3, 5)
a[0], a[1], a[2] = 1, 2, 3

b := append(a, 4)
b[0] = 100

fmt.Println(a)
fmt.Println(b)
```

[100 2 3]

[100 2 3 4]

Change in b call in a

3- is repeated (it is same as Q1)

4-

```
funcs := make([]func(), 0, 3)
for i := 0; i < 3; i++ {
    funcs = append(funcs, func() { fmt.Println(i) })
}
for _, f := range funcs {
    f()
}
```

3

3

3

4-

```
var ErrNotFound = errors.New("not found")
func find() error {
    return fmt.Errorf("lookup failed: %w", ErrNotFound)
}
func main() {
    err := find()
    fmt.Println(err)
    fmt.Println(err == ErrNotFound)
    fmt.Println(errors.Is(err, ErrNotFound))
}
```

lookup failed: not found

false

true

The `find` function wraps the original error `ErrNotFound` using the `%w` verb.

5-

```
defer func() {  
    if r := recover(); r != nil {  
        fmt.Println("Recovered in main:", r)  
    }  
}()  
go func() {  
    panic("goroutine panic!")  
}()  
  
time.Sleep(1 * time.Second)  
fmt.Println("main finished")
```

panic: goroutine panic!

...

The `recover()` function can only catch a panic if it is executed within the same goroutine where the crash occurred. A `recover` call in the main function cannot catch a panic that happens in a separate goroutine; this causes the program to crash (even though `time.Sleep` was used and messages were printed, the panic in the child goroutine remains uncaught by the main `recover`).

6-

```
defer fmt.Println("deferred 1")
defer func() {
    panic("panic in defer")
}()
defer fmt.Println("deferred 3")
fmt.Println("main body")
```

main body

deferred 3

panic: panic in defer

Due to a panic occurring within the second `defer`, the execution of the remaining `defer` statements (such as the first deferred call) may be affected or halted, and the program terminates carrying the new panic.

7-

```
type Data struct {  
    Values []int  
}  
  
func main() {  
    d1 := Data{Values: []int{1}}  
    d2 := Data{Values: []int{1}}  
    fmt.Println(d1 == d2)  
}
```

invalid operation: d1 == d2 (struct containing []int cannot be compared)

Attempting to compare two structures containing slices will result in a compilation error.